Looping Techniques in Python

Last Updated : 12 Jan, 2026

Python provides multiple looping techniques that help iterate over containers like lists, tuples, sets, and dictionaries. These techniques are efficient, save memory and reduce coding effort compared to traditional for and while loops.

Using enumerate()

enumerate() function allows you to loop through a container while getting both the index and the value.

```
words = ['The', 'Big', 'Bang', 'Theory']

for index, value in enumerate(words):
    print(index, value)
```

```
0 The
1 Big
2 Bang
3 Theory
```

Output

```
0 The
1 Big
2 Bang
3 Theory
```

Explanation:

- **words = ['The', 'Big', 'Bang', 'Theory']:** Creates a list of words.
- **for index, value in enumerate(words):** Loops through the list, returning both index and value of each element.

Using zip()

zip() function combines two or more containers and iterates over them in parallel. The loop stops at the length of the shortest container.

```
names = ['Leo', 'Kendall', 'Harry']
ages = (24, 27, 25)

for name, age in zip(names, ages):
    print(f"Name: {name}, Age: {age}")
```

```
Name: Leo, Age: 24
Name: Kendall, Age: 27
Name: Harry, Age: 25
```

Output

```
Name: Leo, Age: 24
Name: Kendall, Age: 27
Name: Harry, Age: 25
```

Explanation:

- **names = ['Leo', 'Kendall', 'Harry']:** Creates a list of names.
- **ages = (24, 27, 25):** Creates a tuple of corresponding ages.
- **for name, age in zip(names, ages):** Loops through both lists simultaneously using zip(), pairing each name with its corresponding age.

Using items()

items() allows you to loop through a dictionary and access both its keys and values simultaneously.

```python
p1 = {'Washington': 'First', 'Lincoln': 'Emancipator', 'Roosevelt': 'Trust-Buster'}

for name, title in p1.items():
    print(name, title)
```

```
Washington First
Lincoln Emancipator
Roosevelt Trust-Buster
```

Output

```
Washington First
Lincoln Emancipator
Roosevelt Trust-Buster
```

Explanation: for name, title in p1.items(): Loops through the dictionary, giving access to each key (name) and value (title).

Using sorted()

sorted() iterates over the list in ascending order without changing the original list.

```python
num = [1, 3, 5, 6, 2, 1, 3]

print("Sorted list:")
for n in sorted(num):
    print(n, end=" ")

print("\nSorted list without duplicates:")
for n in sorted(set(num)):
```

```
    print(n, end=" ")
```

```
Sorted list without duplicates:
1 2 3 5 6
```

Output

```
Sorted list:
1 1 2 3 3 5 6
Sorted list without duplicates:
1 2 3 5 6
```

Explanation:

- **for n in sorted(num):** Loops through the list in ascending order without changing the original list.
- **for n in sorted(set(num)):** Converts the list to a set to remove duplicates, then sorts it.

Using reversed()

reversed() iterates over a container in reverse order without modifying it.

```
num = [1, 3, 5, 6, 2, 1, 3]

for n in reversed(num):
    print(n, end=" ")
```

```
3 1 2 6 5 3 1
```

Output

```
3 1 2 6 5 3 1
```

Explanation: for n in reversed(num): Loops through the list in reverse order without changing the original list.

While Loop with If Condition

while loops are useful when you need conditional iteration.

```
count = 0

while count < 5:
    if count == 3:
        print("Count is 3")
    count += 1
```

```
Count is 3
```

Output

```
Count is 3
```

Explanation:

- **count = 0:** Initializes a variable count to 0.
- **while count < 5:** Runs the loop as long as count is less than 5
- **if count == 3:** Checks if count is equal to 3.
- **print("Count is 3"):** Prints a message when count is 3.
- **count += 1:** Increments count by 1 on each iteration.

Related Articles:

- *Use enumerate() and zip() together in Python*
- *Using Iterations in Python Effectively*

Comment

K kartik 166

Explore

Python Fundamentals

Python Data Structures

Advanced Python

Data Science with Python

Web Development with Python

Python Practice

Courses

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved